Knowing When You're Done: Termination Detection for Recurrent Latent Reasoners under Depth Shift

Md Fahim Sarker
(affiliation and e-mail)

Abstract

Recurrent latent reasoners can solve harder problems by iterating longer, but only if they know when to stop. We study termination detection in a setting where the answer is known. For local recurrent models, a light-cone argument gives each input a lower bound on the step at which its answer can be certified. The bound depends on the input's depth, not its length, and a local automaton attains it at every length. We build DepthShift, a benchmark with exact per-instance depth and splits that vary depth and length independently, and evaluate fifteen stopping rules on one backbone family. Jointly trained pooled gates (ACT, PonderNet, TRM-style heads) leak length: ACT is 99.8% accurate on deeper addition inputs using fewer steps than the bound, through a global shortcut, and 35.6% on inputs that are also longer. Global convergence rules halt before the bound on most deep inputs yet stay correct, so premature stopping and premature error must be measured separately. On addition, per-position confidence halts at exactly the bound on every split, and the local completion heads we introduce match it without improving on it. On segmented parity, every learned signal, completion heads included, learns the training clock and halts at the training depth. Termination detection beyond the training depth is limited less by the readout than by training data that never shows an unfinished computation at a late step. Code, frozen splits and every run are released.

1 Introduction

Recurrent and looped models solve harder problems by thinking longer: the same block is applied again and again, and more iterations buy more reasoning [Schwarzschild et al., 2021b, Bansal et al., 2022, Geiping et al., 2025, Jolicoeur-Martineau, 2025]. The promise is adaptive compute, spending few steps on easy inputs and many on hard ones. Keeping it requires a stopping rule, and the rules in use (learned halting gates [Graves, 2016, Banino et al., 2021], confidence thresholds, and convergence tests [Zhou et al., 2020, Geiping et al., 2025]) are fitted on the difficulty seen in training. The interesting inputs are the ones that are harder than anything seen in training, and there a stopping rule has to decide, from information the model has never been rewarded for reading, that the computation is finished.

We study this decision in a setting where the right answer to "when is it finished?" is known. For a local recurrent reasoner, where each step moves information a fixed number of positions, a light-cone argument gives every input a lower bound $\mathrm{LB}(x)$ on the step at which its answer can be certified (Theorem 1). Halting earlier returns an answer that the model's own information does not determine, which is correct only by luck of the data. The bound depends on the input's *depth*, not its length, so it separates two kinds of distribution

Preprint.

shift that are usually confounded: longer inputs and deeper ones. A local, size-invariant automaton can halt exactly at the bound for every length (Proposition 2), so the bound is achievable, and we can ask which learned or post-hoc rules achieve it.

Contributions.

- **Theory.** A light-cone lower bound for local recurrent reasoners that turns "halting too early" into a per-instance, checkable quantity, with corollaries for local commit-on-done rules and for rules that see only the input length, and closed-form depth oracles plus a matching automaton for addition and segmented parity (Section 2). Every statement has an executable test.
- **Benchmark.** DepthShift: tasks with exact per-instance depth, and frozen splits that vary depth and length independently, up to 12.8× the training depth (Section 3).
- **Evaluation.** Fifteen stopping rules and four references on one backbone family (post-hoc confidence and convergence rules, ACT, PonderNet, TRM-style and v1 pooled heads, and local completion heads that we introduce), scored by accuracy, steps, and premature stops and errors relative to $\mathrm{LB}(x)$ (Sections 4 and 5).

Findings. (i) Jointly trained pooled gates leak length: on addition, ACT is 99.8% accurate on deeper inputs while using fewer steps than the bound allows, by reading a global shortcut, and falls to 35.6% once inputs are both deeper and longer. (ii) Premature stopping and premature error are different things: global convergence rules halt before the bound on most deep addition inputs and are still right, because a travelling carry keeps the state moving. (iii) When pending positions are genuinely ambiguous, as in addition, per-position confidence is already a certificate: max-softmax halts at exactly $\mathrm{LB}(x)$ on every split, and our completion heads match it without improving on it. (iv) When they are not, as in parity, every learned signal we tried, including completion heads designed after the automaton, learns the training clock and halts at the training depth. Taken together, the results say that the difficulty of termination detection lies less in the readout architecture than in training data that never shows a computation unfinished at a late step.

2 Setup and theory

Local recurrent reasoners. An input is a sequence $x = (x_0, \ldots, x_{P-1})$ over an alphabet Σ, and the task assigns an output $y_i(x) \in \Gamma$ to every position. Positions outside $\{0, \ldots, P-1\}$ hold a fixed padding symbol $\perp$; we write $\tilde{x}$ for the padded sequence. A *local recurrent reasoner* with growth rate r keeps one latent vector per position and computes

$$e_i = E(x_i), \quad h_i^{(0)} = H(e_i), \quad h_i^{(t+1)} = U\big(h_{i-r:i+r}^{(t)},\, e_{i-r:i+r}\big), \quad \hat{y}_i^{(t)} = D\big(h_i^{(t)}\big), \qquad (1)$$

where E, H, D are pointwise, U is shared across positions and steps, and h_j, e_j outside the input take fixed padding values. Our backbone (Section 4) is an instance with r kernel-3 convolutions per step. Write $W_i^{\rho}(x) = (\tilde{x}_{i-\rho}, \ldots, \tilde{x}_{i+\rho})$ for the window of radius ρ around i, including padding symbols.

Lemma 1 (Light cone)**.** *For every $t \geq 0$, $h_i^{(t)}$ is a function of $W_i^{rt}(x)$ only, and this function does not depend on i. Hence if $W_i^{rt}(x) = W_{i'}^{rt}(x')$ then $\hat{y}_i^{(s)}(x) = \hat{y}_{i'}^{(s)}(x')$ for all $s \leq t$.*

Per-instance depth. The window is all a position can have seen, so the right notion of difficulty is how large a window must be before the answer is pinned down.

Definition 1 (Determining radius and depth)**.** Position i of x is *ρ-determined* if every pair (x', i') with $W_{i'}^{\rho}(x') = W_i^{\rho}(x)$ has $y_{i'}(x') = y_i(x)$. The determining radius $\rho_i(x)$ is the smallest such ρ, and the depth of the instance is $d^*(x) = \max_i \rho_i(x)$.

Because windows are compared at any position i', the definition matches what a translation-equivariant model can know: it can see the input boundary only once $\perp$ enters its window.

Theorem 1 (Light-cone lower bound and unsoundness)**.** *Let f be any local recurrent reasoner with growth rate r, and let $t < \lceil \rho_i(x)/r \rceil$. Then there is a pair (x', i') with $\hat{y}_{i'}^{(t)}(x') = \hat{y}_i^{(t)}(x)$*

but $y_{i'}(x') \neq y_i(x)$. So at step t the model is wrong at position i of x or at position i' of x'. In particular, no local reasoner can be correct on every input at a step earlier than $\mathrm{LB}(x) = \lceil d^*(x)/r \rceil$.

The theorem constrains what any *stopping rule* can achieve: halting x at step $\tau(x) < \mathrm{LB}(x)$ returns an answer that is not certified by the information the model holds. Whether that answer is wrong depends on which member of the pair the data contain. Two consequences make this concrete.

Corollary 1 (Commit-on-done local rules)**.** *Suppose each position i raises a done flag that depends only on its own history $h_i^{(0)}, \ldots, h_i^{(t)}$, and commits $\hat{y}_i^{(t)}$ when it does. If the system is correct on every input, then position i of x commits no earlier than $\lceil \rho_i(x)/r \rceil$, and the input halts no earlier than* $\mathrm{LB}(x)$.

Corollary 2 (Rules that see only the length)**.** *For addition, d^* ranges over $\{1, \ldots, L+1\}$ at length L. A rule whose halting step depends only on L is correct on every input of length L only if $\tau(L) \geq \lceil (L+1)/r \rceil$. It then spends $\lceil (L+1)/r \rceil - \mathrm{LB}(x)$ extra steps on every input; for uniformly random digits $d^* = O(\log L)$ with high probability, so the waste is $\Theta(L)$ steps. A length-only rule fitted on training depths ($\tau(L) < \lceil (L+1)/r \rceil$) is instead wrong on one member of every flip pair with $d^* > r\,\tau(L)$.*

The bound is achievable with local, size-invariant signals. For our two exact tasks, the determining radius has a closed form, and a local automaton reaches the bound for every input length.

Proposition 1 (Depth oracles)**.** Addition *(digits least-significant first, outputs at positions $0, \ldots, L$ with position L the carry): let a* stop *be a position $j < L$ with $a_j + b_j \neq 9$. Then $\rho_i(x) = i - s_i$, where s_i is the nearest stop before i, or -1 if there is none. Thus $d^*(x)$ is the largest gap between consecutive elements of $\{-1\} \cup \{stops\} \cup \{L\}$.* Segmented parity*: $\rho_i(x) = i - s_i$ for non-delimiter i, where s_i is the previous delimiter or -1; $d^*(x)$ is the longest run of non-delimiters.*

Proposition 2 (Achievability)**.** *For addition and segmented parity there is a local reasoner with $r = 1$ and a commit-on-done local flag such that, for every input of every length, every committed answer is correct and the input halts at exactly $\mathrm{LB}(x) = d^*(x)$ steps.*

The automaton behind Proposition 2 stores one bit of *pending* state per position (is my carry-in, or my segment prefix, known yet?) and flips it when a resolved neighbour passes its value along. Its done flag is a pointwise function of its state, and the same rule works at every length. This is the inductive bias our completion heads target. A pooled gate or a global confidence statistic, in contrast, has no reason to wait for the slowest pending chain. Section 5 tests whether learned signals behave like the automaton beyond the training depth.

Mazes. For mazes the determining radius has no closed form, and information must travel along corridors rather than straight lines, so the light cone gives only a loose bound. We use the optimal path length and the dead-end-filling depth as empirical depth proxies.

All proofs are in Section A. Each result has an executable check in `tests/test_theory.py`: the flip test for Lemma 1 on random backbones, the pair construction of Theorem 1, brute-force verification of Proposition 1, and the automata of Proposition 2 on lengths up to 512.

3 The DepthShift benchmark

DepthShift pairs tasks that have an exact per-instance depth oracle with a split design that separates *longer* inputs from *deeper* ones (Figure 1). All generators run on the GPU from fixed seeds; test sets are frozen and stored with SHA-256 hashes, and the training sampler drops any instance whose hash matches an in-distribution validation or test instance.

T1: addition. Two L-digit numbers, least-significant digit first, one token $10a_i + b_i$ per position, plus one position for the final carry. By Proposition 1, d^* is the longest run of digit pairs summing to 9, plus one. To control d^* independently of L, the generator inserts

Table 1: DepthShift splits for addition (T1) and segmented parity (T2). Every OOD split is evaluated once, at the end; thresholds are selected on ID-val.

Split	Length	Depth d^*	Instances	Purpose
Train (sampled online)	4–20	1–10	–	training
ID-val / ID-test	4–20	1–10	2000 / 2000	thresholds / test
OOD-val	24–32	12–16	1000	backbone tuning only
OOD-depth	4–20[a]	11–20	2000	deeper, same length
OOD-length	64–256	1–10	1000	longer, same depth
OOD-both	64–256	32–128	1000	deeper and longer

[a]10–20 for addition and 11–20 for parity, the shortest lengths that admit the depth.

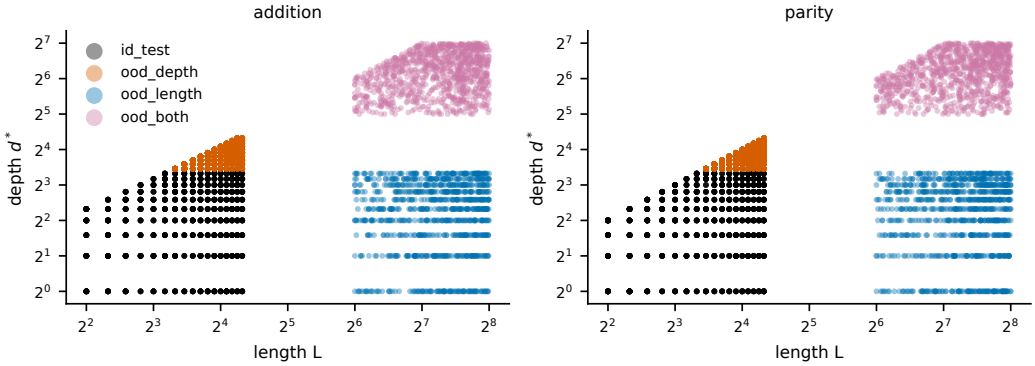


Figure 1: Length and depth of every frozen test instance. OOD-depth and OOD-length differ from ID in one variable each, so length effects and depth effects can be measured separately.

exactly one run of length $d^* - 1$ between two stops, draws background runs with a random density, and caps them below $d^* - 1$. Stops are uniform over the 90 pairs with $a + b \neq 9$, so a run receives a carry with probability 1/2.

T2: segmented parity. Bits with delimiter tokens; the output at each position is the parity of the current segment so far. d^* is the longest segment. Unlike addition, intermediate outputs keep flipping as information arrives, so the per-step predictions do not settle early.

T3: mazes. $n \times n$ mazes from a randomized depth-first generator (input channels: open cell, start, goal; output: the shortest path). Information travels along corridors, so the light cone gives only a loose bound. We record two empirical depth proxies: the optimal path length and the number of dead-end-filling iterations a local rule needs to isolate the path. The generator and splits are released, but within our compute budget no maze backbone passed Gate G1 (Section 7), so the experiments use T1 and T2 only.

Splits. For T1 and T2 (Table 1): in-distribution (ID) lengths 4–20 and depths 1–10; OOD-depth keeps the lengths but uses depths 11–20; OOD-length uses lengths 64–256 at ID depths; OOD-both uses lengths 64–256 and depths 32–128, up to 12.8× the training maximum. A separate OOD-validation split (lengths 24–32, depths 12–16) is used only to tune the backbone. Thresholds of every stopping rule are chosen on ID-validation. For mazes we train on 9×9 and test on 13×13 to 33×33.

4 Backbone and completion heads

Backbone. GLWM-2 is an instance of Equation (1): a pointwise embedding E, a pointwise initial state $H = \mathrm{LN}$, and the shared step

$$h^{(t+1)} = \mathrm{LN}\Big(h^{(t)} + W_{\mathrm{out}}\,\mathrm{GELU}\big(\mathrm{Conv}_3^{(r)} \cdots \mathrm{GELU}(\mathrm{Conv}_3^{(1)}[h^{(t)}; e])\big)\Big), \tag{2}$$

Algorithm 1 Halting with local completion heads (batched; halted instances leave the batch).

Require: input x, backbone (E, H, U, D), completion head c, threshold τ, budget $T_{\max}$

$e \leftarrow E(x)$; $h \leftarrow H(e)$; $p \leftarrow \mathbf{0}$
for $t = 1, \dots, T_{\max}$ **do**
  $h \leftarrow U(h, e)$; $\hat{y} \leftarrow \arg\max D(h)$
  $p \leftarrow \sigma\big(c(h, p)\big)$ ▷ per position; the pointwise head ignores p
  if $\min_{i \text{ valid}} p_i > \tau$ **or** $t = T_{\max}$ **then**
    return $(\hat{y}, t)$
  end if
end for

with r kernel-3 convolutions per step, input re-injection ("recall"), a residual connection, and a LayerNorm over channels at every position. The decoder is a pointwise two-layer MLP. The growth rate is therefore exactly r positions per step (checked by the flip test). We use width 128 and $r = 1$. The backbone is trained with a per-step cross-entropy averaged uniformly over $T_{\text{train}} = 20$ steps ("fixed prior") or with the progressive loss of Bansal et al. [2022]; the pilot (Section 5) selects the objective per task.

Completion heads. A completion head reads each position's latent state and predicts whether that position is *done*: correct now and at every later step. The target is computed from the model's own trajectory,

$$z_i^{(t)} = \prod_{s=t}^{T} \mathbb{1}\big[\hat{y}_i^{(s)} = y_i\big], \tag{3}$$

i.e. a reversed cumulative minimum of per-position correctness, and the head is trained with binary cross-entropy. We study three variants. *Post hoc*: a pointwise MLP on $h_i^{(t)}$, fitted after the backbone is frozen (rollouts to $2T_{\text{train}}$). *Joint*: the same head trained alongside the backbone with an auxiliary binary cross-entropy (weight 1). Unlike ACT and PonderNet, the head never reweights the per-step task loss: the backbone still receives the same task gradient at every step, so the head is a separate prediction problem rather than a halting distribution over the task loss. *Propagating*: the head also reads its neighbours' previous done probabilities through a kernel-3 convolution, $p_i^{(t)} = \sigma\big(g(h_i^{(t)}) + \text{Conv}_3[p^{(t-1)}, \mathbf{1}]_i\big)$, which mirrors the pending-bit automaton of Proposition 2: a position can learn to wait until its left neighbour reports done.

Halting rule. An instance halts at the first step where every valid position is done with probability above τ (Algorithm 1); τ is selected on ID-val as the cheapest threshold whose accuracy is within 0.5 points of the best. Because the rule takes a minimum over positions, its cost grows with the slowest position, not with the input length.

Baselines. All baselines use the same backbone and selection protocol. *References*: fixed T (selected on ID-val), the oracle lower bound $\text{LB}(x)$, the any-step oracle (first step with the right answer), and the stable oracle (first step from which the answer stays right). *Post hoc*: max-softmax and entropy (minimum confidence over positions), k-step prediction stability [Zhou et al., 2020], KL between consecutive steps and hidden-state change (maxima over positions), maximum confidence over steps [Fan et al., 2024], and a pooled correctness head in the style of TRM [Jolicoeur-Martineau, 2025] fitted post hoc. *Jointly trained*: ACT [Graves, 2016], PonderNet with its KL to a geometric prior [Banino et al., 2021], a joint pooled correctness head, and the v1 GLWM pooled gate with its initialisation fixed.

5 Experiments

Protocol. Every rule reads the same frozen backbone checkpoints, except the jointly trained methods, which train their own. Thresholds are selected on ID-val (Section 4) and

Table 2: Addition (T1): accuracy (%) and mean steps of every stopping rule. Rows: references, post-hoc rules, jointly trained pooled gates, completion heads. Two seeds per row; brackets are 95% bootstrap intervals over instances.

	id-test		ood-depth		ood-length		ood-both	
	Acc. (%)	Steps	Acc. (%)	Steps	Acc. (%)	Steps	Acc. (%)	Steps
Fixed T	100.0 [100.0, 100.0]	10.0	70.9 [69.5, 72.3]	10.0	100.0 [100.0, 100.0]	10.0	49.5 [49.5, 49.5]	10.0
Oracle (stable)	100.0 [100.0, 100.0]	2.9	100.0 [100.0, 100.0]	4.9	100.0 [100.0, 100.0]	4.4	99.6 [99.6, 99.7]	40.3
Max-softmax	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.8 [99.8, 99.8]	75.6
Entropy	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.6 [99.4, 99.8]	75.4
k-step stability	99.9 [99.9, 100.0]	3.8	99.9 [99.8, 100.0]	5.2	100.0 [100.0, 100.0]	5.4	99.8 [99.7, 99.9]	40.5
KL between steps	100.0 [100.0, 100.0]	4.0	99.8 [99.7, 100.0]	5.3	100.0 [100.0, 100.0]	5.6	99.8 [99.7, 99.9]	40.6
Hidden-state change	100.0 [100.0, 100.0]	6.2	100.0 [100.0, 100.0]	14.1	69.5 [69.4, 69.6]	4.3	42.7 [42.6, 42.8]	4.2
Max confidence (Fan et al.)	100.0 [100.0, 100.0]	26.0	100.0 [100.0, 100.0]	36.0	100.0 [100.0, 100.0]	26.0	99.8 [99.8, 99.9]	144.0
Correctness head (post hoc)	99.6 [99.4, 99.8]	3.6	99.5 [99.1, 99.9]	5.4	52.4 [51.9, 52.8]	3.3	50.0 [49.6, 50.4]	50.0
ACT	99.9 [99.9, 99.9]	2.4	99.8 [99.8, 99.8]	2.1	43.7 [43.7, 43.7]	2.7	35.6 [35.6, 35.6]	2.9
PonderNet	100.0 [100.0, 100.0]	5.0	99.2 [99.2, 99.2]	6.5	81.3 [81.3, 81.3]	5.3	47.5 [47.5, 47.5]	5.4
Completion head (post hoc)	100.0 [100.0, 100.0]	5.1	100.0 [100.0, 100.0]	13.5	100.0 [100.0, 100.0]	5.5	99.6 [99.4, 99.7]	75.4
Propagating head (post hoc)	100.0 [100.0, 100.0]	5.1	99.8 [99.8, 99.9]	13.5	100.0 [100.0, 100.0]	5.5	99.6 [99.4, 99.7]	75.4

Table 3: Segmented parity (T2), same layout as Table 2. The backbone fails G1 on OOD-both (stable oracle 85%).

	id-test		ood-depth		ood-length		ood-both	
	Acc. (%)	Steps	Acc. (%)	Steps	Acc. (%)	Steps	Acc. (%)	Steps
Fixed T	100.0 [100.0, 100.0]	9.0	30.9 [30.0, 31.8]	9.0	100.0 [100.0, 100.0]	9.0	0.0 [0.0, 0.0]	9.0
Oracle (stable)	100.0 [100.0, 100.0]	3.5	100.0 [100.0, 100.0]	11.8	100.0 [100.0, 100.0]	4.5	85.4 [79.8, 90.9]	90.1
Max-softmax	100.0 [100.0, 100.0]	4.9	51.0 [50.3, 51.8]	10.3	100.0 [100.0, 100.0]	5.4	0.8 [0.4, 1.2]	37.4
Entropy	100.0 [100.0, 100.0]	4.9	35.2 [33.7, 36.8]	9.4	100.0 [100.0, 100.0]	5.4	0.1 [0.0, 0.1]	17.9
k-step stability	100.0 [100.0, 100.0]	4.3	51.0 [45.0, 57.0]	10.7	100.0 [100.0, 100.0]	5.4	4.5 [2.0, 7.0]	54.1
KL between steps	100.0 [100.0, 100.0]	5.9	94.0 [90.6, 97.5]	13.0	100.0 [100.0, 100.0]	6.4	56.6 [35.8, 77.5]	83.9
Hidden-state change	100.0 [100.0, 100.0]	6.7	63.2 [59.2, 67.2]	11.0	98.6 [98.6, 98.7]	6.8	0.0 [0.0, 0.0]	13.2
Max confidence (Fan et al.)	100.0 [100.0, 100.0]	26.0	100.0 [100.0, 100.0]	36.0	100.0 [100.0, 100.0]	26.0	85.2 [79.6, 90.7]	144.0
Correctness head (post hoc)	99.8 [99.7, 100.0]	4.6	32.2 [31.7, 32.6]	9.1	86.1 [84.1, 88.2]	4.9	0.0 [0.0, 0.0]	8.5
Completion head (post hoc)	99.8 [99.6, 100.0]	4.9	30.6 [29.9, 31.2]	9.0	100.0 [99.9, 100.0]	5.4	0.0 [0.0, 0.0]	9.0
Propagating head (post hoc)	100.0 [100.0, 100.0]	4.9	30.6 [30.0, 31.1]	9.0	100.0 [100.0, 100.0]	5.4	0.0 [0.0, 0.0]	9.0

every OOD split is evaluated once. We report exact-match accuracy over all output positions, mean steps, and two error modes measured against the per-instance lower bound $\mathrm{LB}(x)$: the *premature-stop rate* (halting before $\mathrm{LB}(x)$, i.e. returning an uncertified answer) and the *premature-error rate* (halting before $\mathrm{LB}(x)$ *and* being wrong). Brackets are 95% bootstrap intervals over test instances, pooled over two seeds; paired comparisons use Wilcoxon tests across seeds with Holm correction. The backbone objective is fixed-prior for both tasks (the pilot was a tie on validation; `PREREGISTRATION.md`).

Backbones (Gate G1). With the stable oracle as the stopping rule, the addition backbone is correct on 99.6% of OOD-both instances, whose depths reach 12.8× the training maximum (Figure 7); it passes G1. The parity backbone reaches 100% on OOD-depth and OOD-length but only 85% on OOD-both, and its per-step accuracy decays after the peak (Figure 6); it fails G1, so on parity we measure how stopping rules behave on an imperfect backbone. A fixed step count chosen on ID-val is correct on 71% (addition) and 31% (parity) of OOD-depth instances: depth shift needs adaptive halting.

Pooled gates leak length (H1). The jointly trained pooled gates look excellent in distribution and even on OOD-depth, and collapse once inputs get longer (Table 2). ACT (with the tuned $\tau = 0.01$) is correct on 99.8% of OOD-depth instances in about two steps, far below the light-cone bound, yet on 43.7% of OOD-length and 35.6% of OOD-both instances. The model has found a shortcut: it halts at step 1 if no carry can enter the long run of nines and at step 5 otherwise, and the backbone's default output switches from "no carry" to "carry" around step 4. The pooled gate reads a global bit that a single long run makes reliable and many runs make useless. PonderNet and the post-hoc pooled correctness head (in the style of TRM) fail the same way (47.5% and 50.0% on OOD-both). The step counts of all three are nearly flat in depth (Figure 2): they spend what they spent in training.

Premature stopping is not premature error (H2). Global convergence rules (k-step stability and the KL divergence between steps) halt before $\mathrm{LB}(x)$ on 75% of OOD-depth

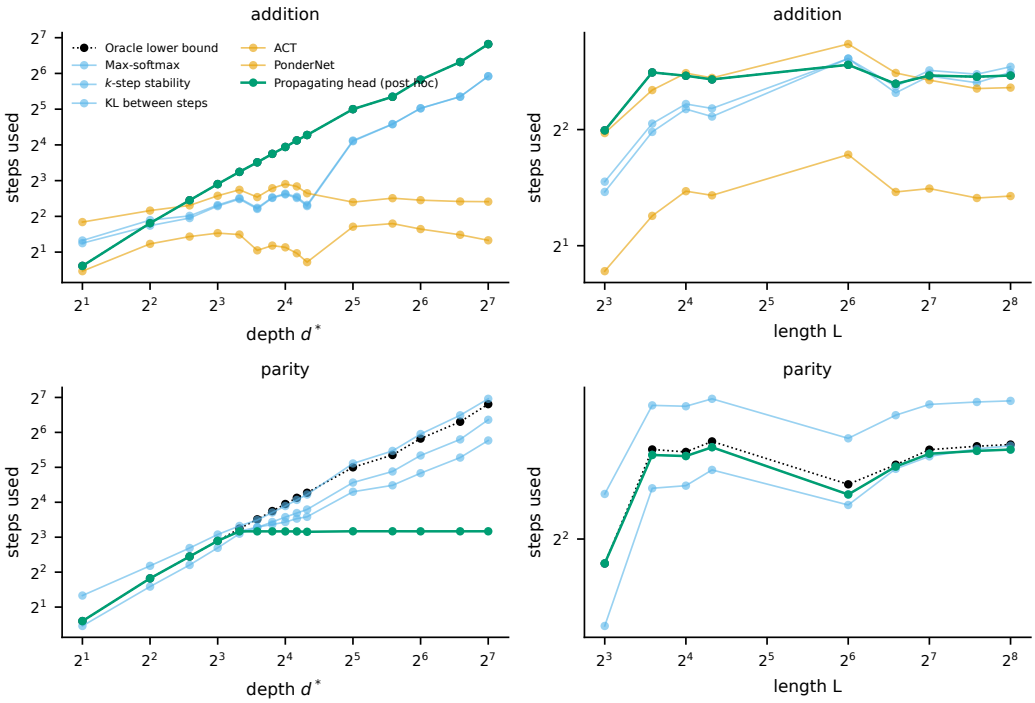


Figure 2: Steps used against depth (left) and length (right). The light-cone bound (dotted) grows with depth only. Per-position rules on addition track it; pooled gates stay flat.

Table 4: Premature-stop rate (%): fraction of instances halted before the light-cone bound LB(x). Such answers are uncertified (Theorem 1), though not necessarily wrong.

	addition		parity	
	OOD-depth	OOD-both	OOD-depth	OOD-both
Fixed T	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Max-softmax	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	90.8 [89.9, 91.7]	98.2 [97.5, 98.8]
Entropy	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	99.2 [98.9, 99.5]	100.0 [100.0, 100.0]
k-step stability	74.7 [73.3, 76.0]	49.7 [47.4, 51.9]	86.2 [85.2, 87.3]	85.5 [84.0, 87.1]
KL between steps	74.7 [73.3, 76.0]	49.7 [47.4, 51.9]	49.2 [47.7, 50.7]	28.5 [26.5, 30.6]
Hidden-state change	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	81.1 [79.9, 82.3]	100.0 [100.0, 100.0]
Max confidence (Fan et al.)	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
Correctness head (post hoc)	75.3 [73.9, 76.6]	99.6 [99.3, 99.9]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
ACT	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	–	–
PonderNet	75.5 [73.5, 77.4]	100.0 [100.0, 100.0]	–	–
Correctness head (joint)	–	–	–	–
v1 GLWM gate	–	–	–	–
Completion head (post hoc)	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Propagating head (post hoc)	0.7 [0.4, 0.9]	0.1 [0.0, 0.1]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Completion head (joint)	–	–	–	–
Propagating head (joint)	–	–	–	–

and 50% of OOD-both addition instances (Table 4), yet remain 99.8% accurate. This is exactly the gap Theorem 1 leaves open: their answers are uncertified, but on addition the data rarely contain the flip partner, and while a carry wave is still travelling the hidden state keeps changing, so convergence is a good proxy for completion. The hidden-state-change rule, whose threshold is set on short inputs, halts after about four steps on OOD-both and is correct on 42.7%. On parity, where outputs keep flipping as information arrives, confidence is misleading: max-softmax halts before the bound on 91% of OOD-depth instances and is correct on 51%; on OOD-both it is correct on 0.8%. Only the KL rule partly survives (94% on OOD-depth, 57% on OOD-both). Both H1 and H2 hold, so Gate G2 passes.

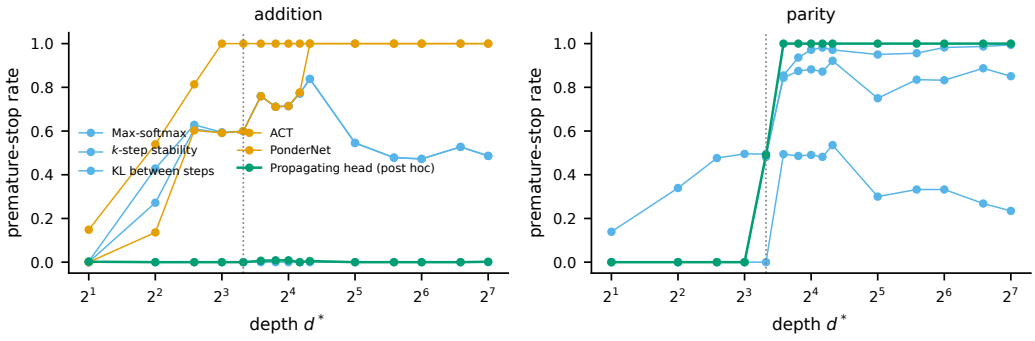


Figure 3: Premature-stop rate against depth; the dotted line is the training maximum.

Per-position certificates on addition (H3). On addition every per-position rule halts at *exactly* the light-cone bound on every split, with no premature stops and full accuracy: max-softmax, entropy, and all four completion-head variants use 13.5 steps on OOD-depth and 75.4 on OOD-both, identical to the oracle LB(x). The reason is visible in the theory: a pending addition digit has two equally plausible outputs (carry or no carry), so a calibrated decoder is uncertain at exactly the positions that are not yet determined, and the minimum over positions is a local certificate in the sense of Corollary 1. A linear probe recovers the pending bit from the hidden state with AUC above 0.999 on every split (**??**). Per-position confidence on addition therefore already implements the automaton of Proposition 2, and a learned completion head adds nothing: G3, which required a 10-point gain over the best baseline, fails on addition by construction.

Completion heads on parity learn the clock. On parity all completion heads halt after about nine steps regardless of depth, close to the largest training depth, and are correct on 31% of OOD-depth and 0% of OOD-both instances. The joint and propagating variants behave the same. In training every position is done by step 10, so "late" and "done" are indistinguishable, and the parity backbone's hidden state encodes elapsed time well enough for the head to use it. The propagating head was designed to wait for its neighbours, but nothing in the training data forces it to: a neighbour is also always done by step 10. This is the same failure as the pooled gates, moved from the input length to the step count. G3 fails on parity as well. Following the pre-registered pivot, we read the results as a map of when simple signals suffice rather than as a win for a new halting method.

Decoupling (H4) and compute. Training a completion head jointly changes the backbone's ID-val step accuracy by less than one point, so the auxiliary target does not distort the task solution. In FLOPs, a completion head costs one pointwise MLP per position and step, and the cost of every rule is dominated by the number of steps it takes (Figure 10). Wall-clock time tells a different story at our scale: with online early exit on 512 OOD-depth instances, checking a halting rule after every step (a host synchronisation plus re-batching) costs more than the steps it saves, and max-softmax and the completion heads are slower than simply running the maximum budget (**??**). Turning certified halting into latency savings needs fused halting checks or larger models whose step cost dominates. The accuracy-versus-steps trade-off of every threshold is shown in Figure 4.

6 Related work

Adaptive computation. ACT [Graves, 2016] and PonderNet [Banino et al., 2021] learn a halting distribution that weights the task loss; Universal Transformers apply ACT per token [Dehghani et al., 2018], and SACT halts spatially [Figurnov et al., 2016]. These gates are trained on the depths seen in training. We evaluate both as baselines on deeper-than-training inputs and separate their sensitivity to length from their sensitivity to depth.

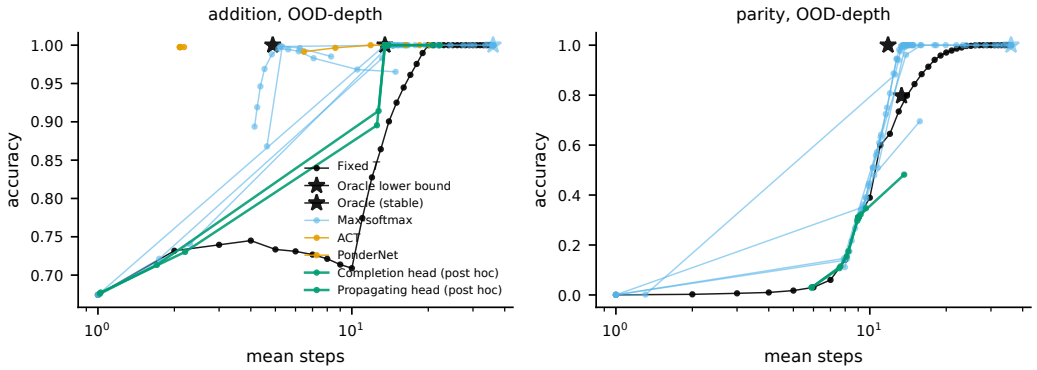


Figure 4: Accuracy against mean steps on OOD-depth for every threshold of every rule.

Recurrent models that extrapolate. Neural GPUs [Łukasz Kaiser and Sutskever, 2015] learn long addition, but fail on long carry chains that random inputs rarely contain [Price et al., 2016]; DepthShift turns this observation into a controlled split. Recurrent networks extrapolate to harder instances by running more iterations [Schwarzschild et al., 2021b,a, Bansal et al., 2022], with the iteration count chosen by the experimenter; path independence makes convergence a usable signal [Anil et al., 2022a], and test-time training can choose per-sample compute [Bao et al., 2025]. Looped Transformers [Giannou et al., 2023, Yang et al., 2023, Saunshi et al., 2025] and the looped models of Fan et al. [2024] need ground-truth step counts during training and stop by oracle or maximum confidence at test time. AdRNNs [Veerabadran et al., 2023] and ANIRA [Moosa et al., 2026] show halting that scales to harder inputs, but harder means larger, so depth and length are confounded. Implicit models [Bai et al., 2019] solve for a fixed point instead of choosing a step.

Latent reasoning and recursive models. Recurrent-depth language models exit per token from the KL between steps [Geiping et al., 2025]; Ouro learns an exit gate [Zhu et al., 2025]; Coconut reasons in continuous latents [Hao et al., 2024]. HRM [Wang et al., 2025] and TRM [Jolicoeur-Martineau, 2025] train a pooled halting or correctness head, and their behaviour has been analysed by Ge et al. [2025]. Several 2026 papers diagnose halting in such models: stochastic stopping schedules and the correlation between loop count and length [Kuo et al., 2026], post-hoc confidence matching learned gates [Popescu et al., 2026], latent motion dropping after the first exact solution [Krishna et al., 2026], and states that encode computational status but become less reliable at larger depth [Liang et al., 2026]. We add a per-instance depth with a provable lower bound, splits that separate depth from length, and a local completion target.

Early exit and knowing when an answer is final. Early-exit networks halt on confidence or patience [Teerapittayanon et al., 2017, Elbayad et al., 2019, Xin et al., 2020, Zhou et al., 2020], with calibrated guarantees in CALM [Schuster et al., 2022]; language models can predict their own correctness [Kadavath et al., 2022]. For reasoning models, early exit uses answer convergence or dedicated predictors [Yang et al., 2025, Liu and Wang, 2025, Nagle et al., 2026, Brown et al., 2026]. These signals are pooled over the whole answer; ours is per position and is tested beyond the training depth.

Length generalisation. Arithmetic and algorithmic tasks are standard probes of length generalisation [Nogueira et al., 2021, Anil et al., 2022b, Delétang et al., 2022, Zhou et al., 2023, McLeish et al., 2024], and shortcut solutions tie the depth a model needs to its architecture [Liu et al., 2022]. We separate depth from length explicitly.

Termination detection. Deciding that a distributed computation has finished from local information is a classic problem [Dijkstra and Scholten, 1980]. Our completion heads learn

Table 5: Closest works by property: per-instance depth (PD), depth/length separation (DL), halting tested deeper than training (OOD-H), local halting signal (Loc), formal stopping result (Th), halting signal decoupled from the task-loss weighting (Dec). Y yes, ∼ partial, – no. Full matrix and search protocol: `paper/related_work_matrix.md`.

Work	PD	DL	OOD-H	Loc	Th	Dec
ACT [Graves, 2016]	–	–	–	∼	–	–
PonderNet [Banino et al., 2021]	–	–	∼	–	–	–
Neural GPU limits [Price et al., 2016]	∼	∼	–	–	–	–
DT-recall [Bansal et al., 2022]	–	–	–	–	–	–
AdRNN [Veerabadran et al., 2023]	∼	–	Y	–	–	–
Looped TF [Fan et al., 2024]	∼	–	∼	–	–	–
Recurrent depth [Geiping et al., 2025]	–	–	–	Y	–	Y
TRM [Jolicoeur-Martineau, 2025]	–	–	–	–	–	∼
ANIRA [Moosa et al., 2026]	∼	–	∼	–	–	–
Stochastic stopping [Kuo et al., 2026]	–	∼	∼	–	–	–
Halting diagnosis [Popescu et al., 2026]	–	–	–	–	–	Y
Finish detection [Krishna et al., 2026]	–	–	∼	–	–	Y
This work	Y	Y	Y	Y	Y	Y

a termination detector for a neural computation, and Proposition 2 shows that a local, size-invariant one exists for our tasks.

7 Limitations

Scale. Every experiment runs on one consumer GPU (RTX 5070, 12 GB). The preregistered grid (five seeds per cell, full maze grid) was cut to fit a five-hour training budget: two seeds per method, three single-seed ablations, and a smaller gate-tuning budget than pre-registered for PonderNet and the v1 gate (`PREREGISTRATION.md`, deviations). Our confidence intervals are bootstrapped over test instances within each run and therefore understate seed variance; several comparisons that we report as ties may separate with more seeds.

Mazes. We could not train a maze backbone that passes Gate G1 within the budget (3% exact match on training-size mazes when stopped), so we make no stopping-rule claims on mazes. Whether local completion heads help when information travels along corridors remains open.

Tasks. The exact theory covers one-dimensional tasks whose determining radius has a closed form. Addition and segmented parity are deliberately simple so that $\mathrm{LB}(x)$ is known for every instance; they are not representative of natural-language reasoning. Our conclusions are about recurrent models with a known, local growth rate; they do not transfer directly to models with global attention, whose light cone covers the whole input after one step.

Backbone dependence. A stopping rule can only be as good as the trajectory it reads. On segmented parity the backbone fails Gate G1 (its stable-oracle accuracy out of distribution is below the preregistered 95%), so parity results measure stopping on an imperfect backbone. The progressive loss was seed-dependent on addition. Termination detection and depth extrapolation are entangled: a rule that looks premature may be reacting to a backbone that never resolves.

What the theory does not say. Theorem 1 rules out *certified* early answers, not correct ones. A rule that halts before $\mathrm{LB}(x)$ can be right on the data distribution, and on addition global convergence detectors often are, because a carry wave keeps the hidden state moving until it lands. We therefore report premature halting and premature *errors* separately, and we do not claim that premature halting is always harmful.

Selection and thresholds. Thresholds are tuned on ID-val, with the cheapest setting within 0.5 points of the best. A different tolerance trades accuracy for steps differently, and

gate hyperparameters (ACT τ, PonderNet λ_p, the v1 gate λ) were tuned on a small grid with one seed. Stronger tuning of the jointly trained gates could narrow some of the gaps we report.

Broader impact. This is basic research on small synthetic tasks. Better termination detection lowers inference compute; we see no direct path to harm beyond that of adaptive-compute models in general.

Acknowledgments and Disclosure of Funding

Compute: one NVIDIA RTX 5070. GPU-hours are reported in Section B.

References

Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, Zico Kolter, and Roger Grosse. Path independent equilibrium models can better exploit test-time computation. arXiv:2211.09961, 2022a.

Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. arXiv:2207.04901, 2022b.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. arXiv:1909.01377, 2019.

Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. arXiv:2107.05407, 2021.

Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end algorithm synthesis with recurrent networks: Logical extrapolation without overthinking. arXiv:2202.05826, 2022.

Hieu Tran Bao, Nguyen Cong Dat, Nguyen Duc Anh, and Hoang Thanh-Tung. Learning to stop overthinking at test time. arXiv:2502.10954, 2025.

Ryan Brown, Zihao Fu, and Chris Russell. Settle: Learning when to stop reasoning. arXiv:2609.38997, 2026.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2018.

Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the chomsky hierarchy. In *International Conference on Learning Representations*, 2022.

Edsger W. Dijkstra and C.S. Scholten. Termination detection for diffusing computations. *Information Processing Letters*, 11(1):1–4, August 1980. ISSN 0020-0190. doi: 10.1016/0020-0190(80)90021-6. URL `http://dx.doi.org/10.1016/0020-0190(80)90021-6`.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. arXiv:1910.10073, 2019.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization. arXiv:2409.15647, 2024.

Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry Vetrov, and Ruslan Salakhutdinov. Spatially adaptive computation time for residual networks. arXiv:1612.02297, 2016.

Renee Ge, Qianli Liao, and Tomaso Poggio. Hierarchical reasoning models: Perspectives and misconceptions. arXiv:2510.00355, 2025.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. arXiv:2502.05171, 2025.

Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. arXiv:2301.13196, 2023.

Alex Graves. Adaptive computation time for recurrent neural networks. arXiv:1603.08983, 2016.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. arXiv:2412.06769, 2024.

Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. arXiv:2510.04871, 2025.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. arXiv:2207.05221, 2022.

Hare Krishna, Shubham Singh, Stephen Ebert, and Hao-Yu Sun. When recursive models finish computing. arXiv:2609.26487, 2026.

Hsun-Yu Kuo, El Mahdi Chayti, Patrik Reizinger, Wieland Brendel, and Martin Jaggi. Stabilizing extrapolation in looped transformers via learned stochastic stopping. arXiv:2606.29983, 2026.

Jia Liang, Xi Jin, and Liangming Pan. Beyond the training horizon: Mechanisms and limits of length generalization in looped transformers. arXiv:2609.33144, 2026.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. arXiv:2210.10749, 2022.

Xin Liu and Lu Wang. Answer convergence as a signal for early stopping in reasoning. arXiv:2506.02536, 2025.

Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers can do arithmetic with the right embeddings. In *Neural Information Processing Systems*, 2024.

Ibraheem Muhammad Moosa, Suhas Lohit, Ye Wang, Moitreya Chatterjee, and Wenpeng Yin. Understanding dynamic compute allocation in recurrent transformers. arXiv:2602.08864, 2026.

Alliot Nagle, Jakhongir Saydaliev, Dhia Garbaya, Michael Gastpar, Ashok Vardhan Makkuva, and Hyeji Kim. Terminator: Learning optimal exit points for early stopping in chain-of-thought reasoning. arXiv:2603.12529, 2026.

Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. Investigating the limitations of transformers with simple arithmetic tasks. arXiv:2102.13019, 2021.

Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Adaptive depth in looped transformers: Diagnosing learned halting gates and trajectory readouts. arXiv:2607.20519, 2026.

Eric Price, Wojciech Zaremba, and Ilya Sutskever. Extensions and limitations of the neural gpu. arXiv:1611.00736, 2016.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. arXiv:2502.17416, 2025.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Neural Information Processing Systems*, 2022.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Arpit Bansal, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. Datasets for studying generalization from easy to hard examples. arXiv:2108.06011, 2021a.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can you learn an algorithm? generalizing from easy to hard problems with recurrent networks. arXiv:2106.04537, 2021b.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. arXiv:1709.01686, 2017.

Vijay Veerabadran, Srinivas Ravishankar, Yuan Tang, Ritik Raina, and Virginia R. de Sa. Adaptive recurrent vision performs zero-shot computation scaling to unseen difficulty levels. arXiv:2311.06964, 2023.

Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. arXiv:2506.21734, 2025.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. Deebert: Dynamic early exiting for accelerating bert inference. arXiv:2004.12993, 2020.

Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models. arXiv:2504.15895, 2025.

Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. arXiv:2311.12424, 2023.

Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Josh Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? a study in length generalization. arXiv:2310.16028, 2023.

Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. Bert loses patience: Fast and robust inference with early exit. arXiv:2006.04152, 2020.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models. arXiv:2510.25741, 2025.

Łukasz Kaiser and Ilya Sutskever. Neural gpus learn algorithms. arXiv:1511.08228, 2015.

A Proofs

Proof of Lemma 1. By induction on t. For $t = 0$, $h_i^{(0)} = H(E(x_i))$ depends on $W_i^0(x) = (x_i)$ through a function that does not depend on i. Assume the claim for t. The update for $h_i^{(t+1)}$ reads $h_j^{(t)}$ and e_j for $|j - i| \leq r$. Each $h_j^{(t)}$ is a fixed function of $W_j^{rt}(x) \subseteq W_i^{r(t+1)}(x)$ (padding positions contribute fixed values, which are also determined by the window because

the window records $\perp$), and each e_j is a function of $\tilde{x}_j \in W_i^r(x)$. Since U is shared, $h_i^{(t+1)}$ is a fixed function of $W_i^{r(t+1)}(x)$. The decoder is pointwise, which gives the statement about $\hat{y}$. □

Proof of Theorem 1. Let $\rho = rt < \rho_i(x)$ (because $t < \lceil \rho_i/r \rceil$ implies $rt < \rho_i$). By minimality of $\rho_i(x)$, position i is not ρ-determined, so there is (x', i') with $W_{i'}^{\rho}(x') = W_i^{\rho}(x)$ and $y_{i'}(x') \neq y_i(x)$. By Lemma 1, $\hat{y}_{i'}^{(t)}(x') = \hat{y}_i^{(t)}(x)$, so this common prediction differs from at least one of the two targets. Taking i with $\rho_i(x) = d^*(x)$ gives the statement about $\mathrm{LB}(x)$. □

Proof of Corollary 1. Suppose position i of x commits at $t < \lceil \rho_i(x)/r \rceil$. Take the pair (x', i') from Theorem 1. The histories $h_i^{(0..t)}(x)$ and $h_{i'}^{(0..t)}(x')$ coincide by Lemma 1, so position i' of x' raises its flag at the same step t (it cannot have raised it earlier, since x would then have raised it earlier too) and commits the same prediction, which is wrong for one of them. The input halts when all positions have committed, so it halts no earlier than $\max_i \lceil \rho_i(x)/r \rceil = \mathrm{LB}(x)$. □

Proof of Corollary 2. At length L the input with $a_j + b_j = 9$ for all $j < L$ has no stops, so $d^* = L + 1$ by Proposition 1, and Theorem 1 applies to the final carry position. Any $\tau(L)$ below $\lceil (L+1)/r \rceil$ is therefore wrong on one member of the corresponding pair, and the same argument applies to every input with $d^*(x) > r\tau(L)$. For uniformly random digits each position is a non-stop with probability $1/10$, independently, so $d^* - 1$ is the longest run of successes in L Bernoulli$(1/10)$ trials, which is $\log_{10} L + O_p(1)$ (the Erdős–Rényi longest-run law). □

Proof of Proposition 1. Addition. Output i is $(a_i + b_i + c_i) \bmod 10$ for $i < L$ and c_L for $i = L$, where c_i is the carry into i. If $s_i \geq 0$ is the nearest stop before i, then positions $s_i + 1, \ldots, i - 1$ all have $a_j + b_j = 9$, so $c_i = c_{s_i+1} = \mathbb{1}[a_{s_i} + b_{s_i} \geq 10]$. If there is no stop, all positions $0, \ldots, i - 1$ sum to 9 and $c_i = c_0 = 0$. *Upper bound* ($\rho_i \leq i - s_i$): the window of radius $i - s_i$ contains position s_i (or the boundary symbol $\perp$ at -1) and every position between, so it fixes c_i and (a_i, b_i), hence y_i, for every (x', i') sharing the window. A boundary symbol at relative offset $-(i - s_i)$ in x' forces $i' = i$, so the carry is 0 there too. *Lower bound* ($\rho_i \geq i - s_i$): take $\rho = i - s_i - 1$. The window contains positions $i - \rho, \ldots, i - 1$, all summing to 9, and no boundary symbol on the left. Build x' by taking the window content, placing it at $i' = \rho + 1$, and putting one digit pair at position 0 of x' whose sum is ≥ 10 if $c_i(x) = 0$ and < 9 if $c_i(x) = 1$ (the right-hand part of the window is copied unchanged). Then $c_{i'}(x') = 1 - c_i(x)$, so $y_{i'}(x') \neq y_i(x)$. *Parity* is identical with "delimiter" for "stop" and the segment prefix parity for the carry: flipping the bit just outside the window changes the parity. □

Proof of Proposition 2. Addition. Each position stores its pair sum class and a carry-in $c_i \in \{0, 1, ?\}$, initially ?. Per step, position i reads its left neighbour's carry-out, which is $\mathbb{1}[a_{i-1} + b_{i-1} \geq 10]$ if $i - 1$ is a stop, c_{i-1} if $a_{i-1} + b_{i-1} = 9$, and 0 if the neighbour is the boundary $\perp$; it sets c_i to that value if $c_i =?$ and the value is known. This update reads only radius-1 neighbours and is the same at every position and length. By induction, c_i becomes known exactly at step $i - s_i = \rho_i$ (step 1 at a stop's right neighbour or at position 0, one more step for each 9 in between), and once known it equals the true carry. The done flag is $\mathbb{1}[c_i \neq ?]$ and the committed output is $(a_i + b_i + c_i) \bmod 10$, which is correct. The input halts at $\max_i \rho_i = d^*(x) = \mathrm{LB}(x)$ for $r = 1$. *Parity*: the same construction with the segment prefix parity in place of the carry, initialised to 0 at delimiters. □

B Implementation details

Training. AdamW, learning rate 10^{-3}, weight decay 10^{-4}, 300 warm-up steps then cosine decay, gradient clipping at 1.0, fp32 with TF32 matrix multiplies (faster than bf16 autocast for this launch-bound model on our GPU). T1/T2: width 128, $r = 1$, batch 512, 8000 iterations, $T_{\text{train}} = 20$. T3: width 64, $r = 4$, batch 32 (with random dihedral augmentation),

5000 iterations, $T_{\text{train}} = 32$. Training batches are sampled on the GPU and filtered against the ID-val and ID-test hashes. Seeds 0–4 for the base models and 0–2 for the jointly trained methods (T1/T2); seeds 0–1 for mazes.

Gate hyperparameters. Every jointly trained gate gets the same budget of three settings on addition (seed 0), and the setting with the fewest mean ID-val steps among those within 0.5 points of the best ID-val accuracy is used everywhere (`configs/tune_gates.yaml`). Settings: ACT ponder weight $\tau \in \{10^{-3}, 10^{-2}, 10^{-1}\}$; PonderNet $(\beta, \lambda_p) \in \{(0.01, 0.1), (0.01, 0.2), (0.1, 0.05)\}$; v1 gate compute weight $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}\}$.

Post-hoc heads. Fitted on the frozen backbone for 1500 iterations (batch 256; mazes: batch 32) on fresh training-distribution batches rolled out to $2T_{\text{train}}$ steps (backbone in bf16 autocast, heads in fp32); Adam, learning rate $2 \cdot 10^{-3}$ with cosine decay. The pooled correctness head (TRM-style) predicts exact match from the masked mean of the latent state; the completion heads predict Equation (3) per position.

Threshold grids. Probability rules: $\{0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.98, 0.99, 0.995, 0.999, 0.9999\}$; entropy, KL and hidden-state change: 19 log-spaced values in $[10^{-6}, 1]$; stability $k \in \{2, 3, 5\}$; ACT cumulative halting $\{0.5, 0.7, 0.9, 0.95, 0.99\}$; PonderNet cumulative halting $\{0.3, 0.5, 0.7, 0.9, 0.95, 0.99\}$; fixed $T \in \{1, \ldots, T_{\max}\}$. Test-time budget $T_{\max} = \lceil \max d^*/r \rceil + 16$ per split.

Metrics. Premature-stop rate: share of instances halted before $\text{LB}(x)$; premature-error rate: share halted before $\text{LB}(x)$ *and* wrong; overthinking: steps beyond the first step from which the answer stays right, on correctly answered instances; regret: accuracy lost and extra steps spent relative to the stable oracle (steps on instances the oracle gets right); depth tracking: Spearman $\rho(\text{steps}, d^*)$; length leakage: partial Spearman $\rho(\text{steps}, L \mid d^*)$; calibration: 15-bin ECE of per-position done probabilities.

Compute. Table 6 lists measured GPU time per sweep.

Table 6: Measured GPU time per sweep (RTX 5070; several runs shared the GPU, so wall-clock per run is higher than in isolation). Runs listed in more than one sweep are counted once per sweep.

Sweep	Runs	Train (h)	Eval (h)
`main_addition`	2	0.3	0.1
`main_gates`	2	0.9	0.3
`main_parity`	2	0.7	0.6
`pilot_addition`	8	2.4	1.1
`tune_gates`	2	1.1	0.2
Total	16	5.4	2.4

C Additional results

The v1 model, re-evaluated. The original GLWM paper reported a mean of 1.05 steps with a gate initialised to halt early. Re-running it with the evaluation of this paper (deduplicated test set, exact match over all positions, Table 7) shows that both initialisations reach full accuracy, the v1 initialisation uses 2.6 steps on average, and only 84% of instances are already correct after one step (Figure 5). The earlier step count came from a halting criterion that stopped before the answer had settled.

Per-step accuracy of the backbones. Figure 6 shows exact-match accuracy at every step for both objectives. On addition, accuracy on every split rises to its plateau and stays there: once an answer is right it stays right. On parity, OOD-both accuracy peaks and then

Table 7: The v1 GLWM model with a fixed gate initialisation versus the original (v1) initialisation, evaluated on the deduplicated 4–8 digit test set.

Gate init	Accuracy	Mean steps	Steps (4-digit)	Acc. at step 1	MFLOPs/sample
fixed	1.0000 ± 0.0000	3.41	3.87	0.5888	7.50
v1	1.0000 ± 0.0000	2.60	3.05	0.8405	7.02

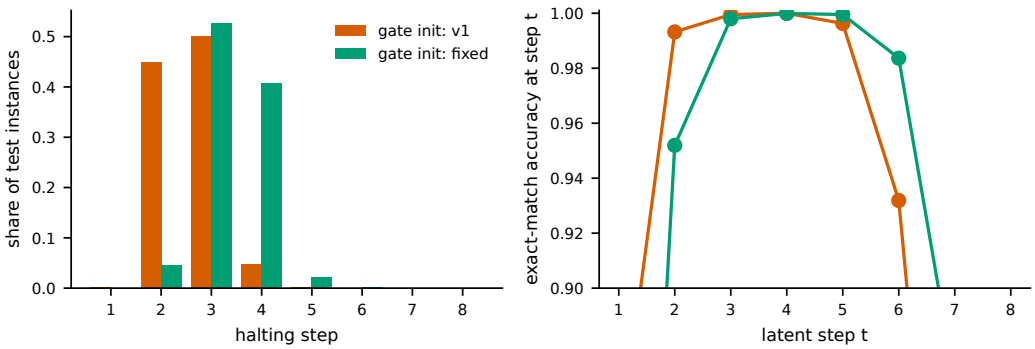


Figure 5: v1 GLWM re-evaluated: halting steps (left) and exact-match accuracy at each latent step (right) for the two gate initialisations.

decays for both objectives, so the stable oracle sits below 100% and the backbone fails Gate G1.

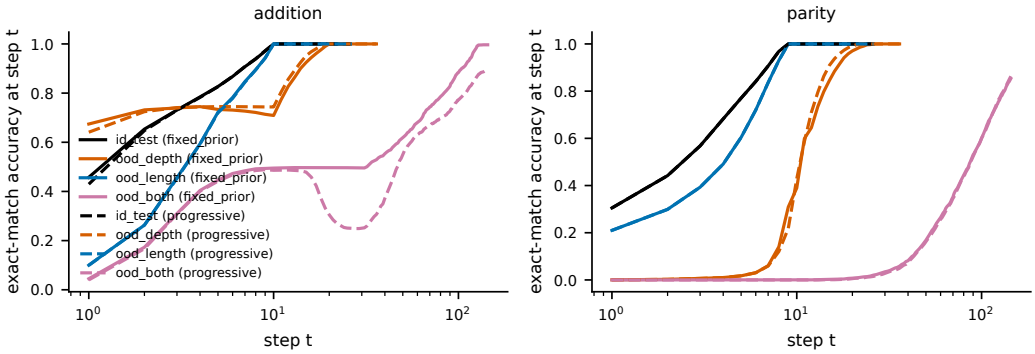


Figure 6: Exact-match accuracy at step t (mean over pilot seeds), fixed-prior (solid) and progressive (dashed) objectives.

Information travels at the light-cone speed. Figure 8 shows, for one OOD-both addition instance, which output positions are correct at which step, together with the per-position lower bound ρ_i from Proposition 1. No position can be certified before its bound, and the backbone's positions settle along a wave that follows it, travelling one position per step from each stop. Figure 9 shows the done probabilities of the two post-hoc completion heads on a deep instance: they switch on along the same wave, after the bound.

Linear probes. A linear probe fitted on ID-val hidden states decodes whether a position is still pending ($t < \rho_i$) from $h_i^{(t)}$, and transfers to OOD-depth and OOD-both without refitting (**??**). The backbone therefore represents the pending bit of the automaton in Proposition 2; whether a stopping rule uses it is a separate question.

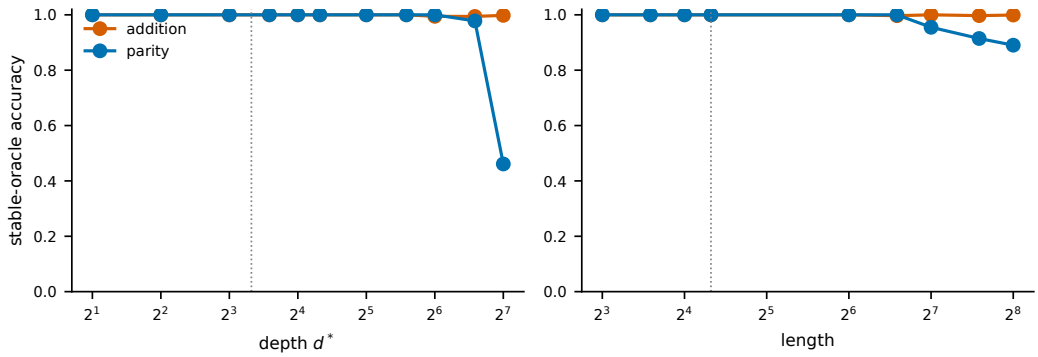


Figure 7: Stable-oracle accuracy against depth (left) and length (right), pooled over splits; dotted lines mark the training maxima. Length alone costs nothing; accuracy on parity drops with depth.

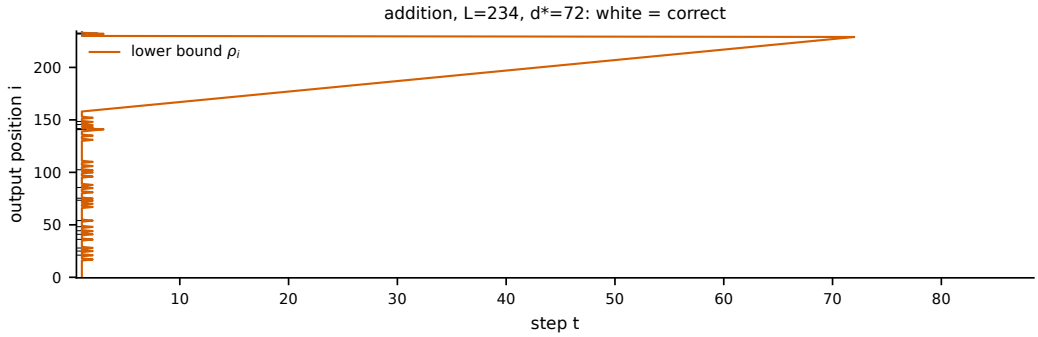


Figure 8: Correctness of every output position (rows) at every step (columns) for one OOD-both addition instance; white is correct. The orange line is the per-position lower bound ρ_i.

Measured compute. Figure 10 plots measured FLOPs per instance (from `FlopCounterMode`, at the mean length of each split) against accuracy. The completion heads add one small MLP per position and step; their cost is dominated by the number of steps they choose.

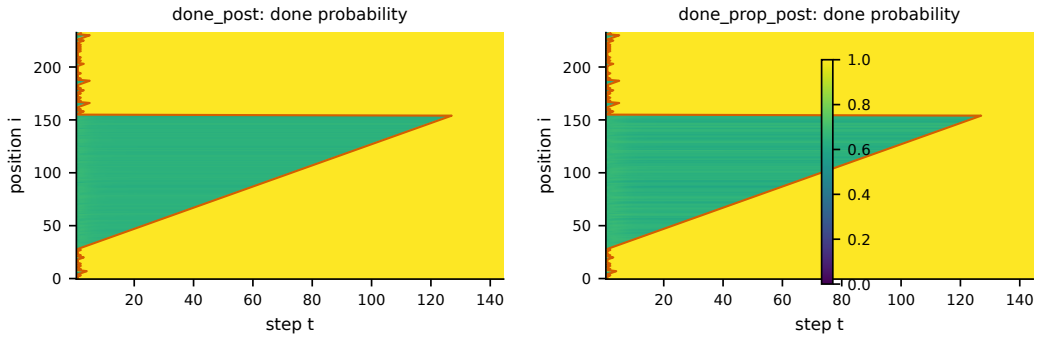


Figure 9: Done probabilities of the post-hoc completion heads (pointwise, left; propagating, right) on a deep OOD-both addition instance, with the per-position lower bound in orange.

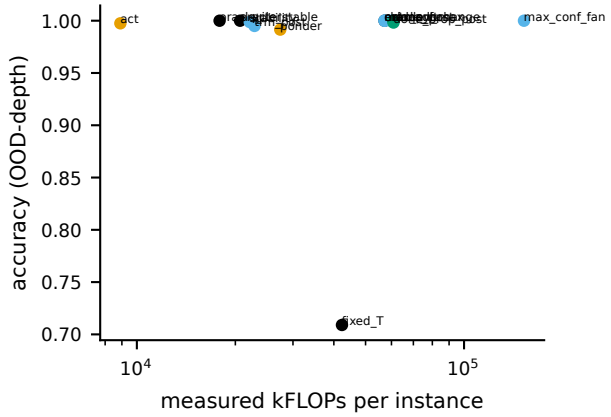


Figure 10: Accuracy on addition OOD-depth against measured kFLOPs per instance for every stopping rule.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state each claim with the experiment that tests it; `paper/claims_map.md` maps every number to its source file.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 discusses the tasks, the architecture class, the loose maze bound, and the cases where simple confidence rules suffice.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Assumptions are stated in Section 2; full proofs are in Section A, and each result has an executable check in `tests/test_theory.py`.

   Guidelines:

   - The answer NA means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section B lists every hyperparameter, split and selection rule; one YAML config per run and `REPRODUCE.md` give the exact commands.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All code, configs, frozen-split hashes and the analysis notebooks are released; the data are generated from fixed seeds.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected

simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Sections 3 and 4 and Section B give splits, objectives, optimiser, schedules, threshold grids and the ID-val selection rule.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: 95% bootstrap confidence intervals over instances (clustered by seed), Wilcoxon tests across seeds and Holm correction; Section 5.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: One RTX 5070; GPU-hours per sweep are reported in Section B.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses synthetic data only and follows the NeurIPS Code of Ethics.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

    Justification: Foundational research on halting in small synthetic-task models; we see no direct societal impact beyond general efficiency of iterative models.

    Guidelines:

    - The answer NA means that there is no societal impact of the work performed.
    - If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out.

For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: No pretrained language or image models or scraped data are released.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: PyTorch (BSD) and the arXiv metadata used for the bibliography are credited; maze data are generated by our own code.

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The DepthShift generators and stopping-rule library are documented in the repository README and `REPRODUCE.md`.

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: No crowdsourcing or human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: No human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: An AI coding assistant was used to write code and draft text under the author's direction; all results come from the released code and were checked by the author. LLMs are not part of the method.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.